

Utilizando *listas*

A **lista** é a coleção mais usada do Python: guarda vários valores, em ordem, e pode ser alterada. Nesta aula: criar, acessar por índice e fatia, adicionar e remover itens, ordenar, buscar, a armadilha de "copiar" com `=`, listas dentro de listas e uma lista de compras montada com `input()`.

[]

Índices e fatias

append · insert · extend

remove · pop · del

sort · sorted

.copy()

Listas aninhadas

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Lists, More on Lists; Built-in Types: Sequence Types; Sorting HOW TO), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Vários valores *num nome só*

criar.py

```
frutas = ["maçã", "banana", "uva"]
numeros = [10, 20, 30, 40]
misturada = ["Ana", 29, 1.68, True]    # pode misturar tipos
vazia = []
print(frutas)
print(len(numeros), len(vazia))
print(frutas[0], frutas[-1])
print(numeros[1:3])
```

SAÍDA

```
['maçã', 'banana', 'uva']
4 0
maçã uva
[20, 30]
```

- ✓ Colchetes, itens separados por vírgula
- ✓ Pode misturar tipos, mas o normal é uma lista de coisas do mesmo tipo
- ✓ Índices e fatias funcionam **igual às strings** (aula 07): começa em 0, -1 é o último
- ✓ `len()` conta os itens

A grande diferença para a string: a lista é **mutável**. Dá para trocar, incluir e tirar itens na própria lista.

Adicionando *e removendo*

alterar.py

```
frutas = ["maçã", "banana", "uva"]
frutas[1] = "manga"           # listas são mutáveis
frutas.append("kiwi")         # adiciona no fim
frutas.insert(0, "pera")      # adiciona na posição 0
frutas.extend(["limão", "caju"]) # adiciona vários
print(frutas)
frutas.remove("uva")          # remove pelo valor
ultima = frutas.pop()         # remove e devolve o último
print(ultima, frutas)
del frutas[0]                 # remove pela posição
print(frutas)
```

SAÍDA

```
['pera', 'maçã', 'manga', 'uva', 'kiwi', 'limão', 'caju']
caju ['pera', 'maçã', 'manga', 'kiwi', 'limão']
['maçã', 'manga', 'kiwi', 'limão']
```

OPERAÇÃO	FAZ
lista[i] = x	Troca o item da posição i
.append(x)	Adiciona um item no fim
.insert(i, x)	Adiciona na posição i
.extend(outra)	Adiciona todos os itens de outra lista
.remove(x)	Tira a 1ª ocorrência do valor x
.pop()	Tira e devolve o último (ou o da posição i)
del lista[i]	Tira pela posição

sort() ou sorted()?

ordenar.py

```
notas = [7.5, 9.0, 6.0, 8.5]
print(sorted(notas))           # lista nova, ordenada
print(notas)                   # a original não mudou
notas.sort(reverse=True)       # ordena a própria lista
print(notas)
print(max(notas), min(notas), sum(notas) / len(notas))
nomes = ["bia", "Ana", "carlos", "Bruno"]
print(sorted(nomes))           # maiúsculas vêm antes
print(sorted(nomes, key=str.lower)) # alfabética de verdade
```

SAÍDA

```
[6.0, 7.5, 8.5, 9.0]
[7.5, 9.0, 6.0, 8.5]
[9.0, 8.5, 7.5, 6.0]
9.0 6.0 7.75
['Ana', 'Bruno', 'bia', 'carlos']
['Ana', 'bia', 'Bruno', 'carlos']
```

	SORTED(LISTA)	LISTA.SORT()
Original	Fica igual	É alterada
Devolve	Uma lista nova	None

- ✓ `reverse=True`: do maior para o menor
- ✓ Maiúsculas vêm antes das minúsculas; `key=str.lower` corrige
- ✓ `max`, `min` e `sum` funcionam direto na lista

`notas = notas.sort()` apaga a lista: `.sort()` devolve `None`.

Procurando *na lista*

buscar.py

```
cores = ["azul", "verde", "azul", "rosa"]
print("verde" in cores)      # True
print(cores.index("azul"))   # 0: primeira posição
print(cores.count("azul"))   # 2
for i, cor in enumerate(cores):
    print(i, cor)
```

SAÍDA

```
True
0
2
0 azul
1 verde
2 azul
3 rosa
```

- ✓ `in` : o item está na lista?
- ✓ `.index(x)` : em que posição (a primeira)? Dá erro se não existir
- ✓ `.count(x)` : quantas vezes aparece
- ✓ `enumerate()` dá posição e valor juntos; o `for` é o assunto da aula 14

= não copia *a lista*

`copia.py`

```
original = [1, 2, 3]
mesmo = original # NÃO copia: mesma lista, dois nomes
copia = original.copy() # cópia de verdade (ou original[:])
mesmo.append(4)
print(original) # [1, 2, 3, 4]: mudou junto!
print(copia) # [1, 2, 3]
```

SAÍDA

```
[1, 2, 3, 4]
[1, 2, 3]
```

MESMO = ORIGINAL

Dois nomes, **uma** lista na memória



ORIGINAL.COPY()

Uma lista **nova**, com os mesmos itens

Esse é um dos bugs mais comuns de iniciante: alterar a "cópia" e ver a original mudar. Com strings isso não acontece porque elas não podem ser alteradas.

Para listas com listas dentro, `.copy()` copia só o primeiro nível. A cópia completa é `copy.deepcopy()`.

Listas *dentro de listas*

aninhadas.py

```
alunos = ["Ana", 9.0], ["Bruno", 7.5], ["Carla", 8.0]
print(alunos[1])          # ['Bruno', 7.5]
print(alunos[1][0])       # Bruno
for nome, nota in alunos:
    print(f"{nome}: {nota}")
```

SAÍDA

```
['Bruno', 7.5]
Bruno
Ana: 9.0
Bruno: 7.5
Carla: 8.0
```

- ✓ Cada item pode ser outra lista: uma tabela com linhas
- ✓ Dois índices: `alunos[1][0]` = linha 1, coluna 0
- ✓ `for nome, nota in alunos` separa cada par (as tuplas da aula 10 explicam o desempacotamento)

Para dados com nome de campo ("nome", "nota"), o dicionário (aula 12) costuma ser mais claro que posições.

Posição *que não existe*

erro_indice.py

```
frutas = ["maçã", "banana", "uva"]  
print(frutas[3])
```

```
Traceback (most recent call last):  
  File "erro_indice.py", line 2, in <module>  
    print(frutas[3])  
          ~~~~~^^^  
IndexError: list index out of range
```

- ✓ 3 itens: posições 0, 1 e 2
- ✓ Mesmo erro "de um a mais" das strings (aula 07)
- ✓ Para acrescentar no fim, use `.append()`, não `lista[3] = ...`
- ✓ `.remove()` de um valor que não está na lista dá `ValueError`

Uma lista *de compras*

compras.py

```
compras = []
compras.append(input("1º item: ").strip())
compras.append(input("2º item: ").strip())
compras.append(input("3º item: ").strip())
compras.sort()
print(f"{len(compras)} itens:", ", ", ".join(compras))
```

SAÍDA (DIGITOU PÃO, ARROZ E LEITE)

```
1º item: pão
2º item: arroz
3º item: leite
3 itens: arroz, leite, pão
```

- ✓ Começa vazia e cresce com `.append()`
- ✓ `.sort()` põe em ordem alfabética
- ✓ `", ".join()` monta a frase final (aula 05)

Com o `while` (aula 15), a lista aceitará quantos itens o usuário quiser, até ele teclar Enter sem digitar nada.

Estatísticas *de uma turma*

Enunciado: com a lista de notas, mostre as notas ordenadas, a média com 2 casas, a maior e a menor, a amplitude e, depois de incluir a nota 8.0 de um aluno novo, quantas notas há.

Uma resposta possível

```
# Exercício: estatísticas de uma turma
notas = [8.5, 6.0, 9.5, 7.0, 5.5, 10.0]
media = sum(notas) / len(notas)
print("Notas ordenadas:", sorted(notas))
print(f"Média: {media:.2f}")
print("Maior e menor:", max(notas), min(notas))
print("Amplitude:", max(notas) - min(notas))
notas.append(8.0)
print("Com a nota do aluno novo:", len(notas), "notas")
```

SAÍDA

```
Notas ordenadas: [5.5, 6.0, 7.0, 8.5, 9.5, 10.0]
Média: 7.75
Maior e menor: 10.0 5.5
Amplitude: 4.5
Com a nota do aluno novo: 7 notas
```

- ✓ Média = `sum` dividido por `len`
- ✓ `sorted()` mostra ordenado sem mexer na original
- ✓ Desafio: mostre a mediana (o valor do meio da lista ordenada)

Teste o que *ficou*

1. Qual adiciona um item no fim da lista?

A) .add()

B) .append()

C) .insert()

D) .push()

B. .insert() é para uma posição.

2. O que `notas.sort()` devolve?

A) A lista ordenada

B) None

C) Uma cópia

D) True

B. Ele ordena a própria lista.

3. `b = a; b.append(9)`. O que acontece com `a`?

A) Nada

B) Também ganha o 9

C) Erro

D) Fica vazia

B. Use `a.copy()` para copiar.

4. Em `[[1, 2], [3, 4]]`, como pegar o 3?

A) `lista[3]`

B) `lista[1][0]`

C) `lista[0][1]`

D) `lista[2]`

B. Linha 1, coluna 0.

Uma coleção *que muda*

Listas guardam vários valores em ordem, com os mesmos índices e fatias das strings, mas são **mutáveis**: `append`, `insert`, `remove`, `pop`, `sort`. `sorted()` devolve uma nova; `=` não copia (use `.copy()`).

Mutável

`append` · `pop`

`sort` × `sorted`

`.copy()`

`lista[i][j]`

Próxima aula → 10 · Utilizando uma Tupla

Fontes: docs.python.org/3/tutorial/introduction.html#lists; docs.python.org/3/tutorial/datastructures.html; docs.python.org/3/howto/sorting.html; docs.python.org/3/library/copy.html, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.